

```

import { q, one, j } from '../db.js';
import { emit } from './events.js';

// A module never gets a connection string and never issues DDL.
// It gets this object, scoped to one package and one business.
// Every row it touches is stamped with that business.
export function makeGateway({ ctx, packageKey }) {
  const businessId = ctx.businessId;

  async function physical(logicalName) {
    const row = await one(
      `select pt.physical_name
        from provisioned_table pt
       join install i on i.id = pt.install_id
      where i.business_id = $1 and pt.package_key = $2 and pt.logical_name = $3
        and i.status = 'active'`,
      [businessId, packageKey, logicalName]
    );
    if (!row) throw new Error(`${packageKey} has no provisioned table "${logicalName}"`);
    return row.physical_name;
  }

  return {
    businessId,
    packageKey,

    async insert(logicalName, values) {
      const table = await physical(logicalName);
      const cols = ['business_id', ...Object.keys(values)];
      const params = [businessId, ...Object.values(values)];
      const marks = params.map((_, i) => `${$${i + 1}}`);
      return one(
        `insert into "${table}" (${cols.map(c => `"${c}"`).join(',')})
         values (${marks.join(',')}) returning *`, params
      );
    },

    async select(logicalName, { where = {}, orderBy = null, limit = 200 } = {}) {
      const table = await physical(logicalName);
      const params = [businessId];
      const clauses = ['business_id = $1'];
      for (const [k, v] of Object.entries(where)) {
        params.push(v);
        clauses.push(`${k} = ${params.length}`);
      }
    }
  };
}

```

```

    }
    const order = orderBy ? ` order by "${orderBy}"` : '';
    return q(`select * from "${table}" where ${clauses.join(' and ')}${order} l
  },

  async update(logicalName, id, values) {
    const table = await physical(logicalName);
    const params = [];
    const sets = Object.entries(values).map(([k, v]) => { params.push(v); retur
    params.push(id, businessId);
    return one(
      `update "${table}" set ${sets.join(',')}
        where id = ${params.length - 1} and business_id = ${params.length} re
    );
  },

  async remove(logicalName, id) {
    const table = await physical(logicalName);
    return one(`delete from "${table}" where id = $1 and business_id = $2 retur
  },

  // Canonical business data is read-only to modules. Changing it is a capabili
  async canonical() {
    const business = await one(`select * from business where id = $1`, [busines
    const locations = await q(`select * from location where business_id = $1`,
    const facts = await q(
      `select key, value from business_fact
        where business_id = $1 and (effective_to is null or effective_to > now(
    );
    const hours = await q(`select * from regular_hours where business_id = $1 o
    const temporary = await q(
      `select * from temporary_hours where business_id = $1 and ends_at > now()
    );
    return {
      business, locations, hours, temporary,
      facts: Object.fromEntries(facts.map(f => [f.key, j(f.value)])),
    };
  },

  async emit(topic, payload) {
    return emit({ businessId, topic: `${packageKey}.${topic}`, payload });
  },

  async settings() {
    const row = await one(
      `select settings from install where business_id = $1 and package_key = $2
    );

```

```
        return j(row?.settings) ?? {};  
    },  
    };  
}
```